```
      Opened(state.status.marketId, command.fixture, command.odds)   ❶
    Effect
      .persist(opened)                                               ❷
      .thenReply(command.replyTo)(_ => Accepted)                     ❸
  }
  ...
}
```

❶ Event creation

❷ Event persisting in the journal

❸ Reply to the sender

Each of the command handler methods includes a variation of this structure. The command creates an event, persists it, and finally responds to the caller. Not every command handler methods has persistence or a response, but their structure is basically the same.

The list of events is the effect of a command with the same root name. The command `Open` creates an event `Opened`, and so on. The following snippet shows the list of events:

```
object Market {
  ...

  sealed trait Event extends CborSerializable {
    def marketId: String
  }
  final case class Opened(marketId: String, fixture: Fixture, odds: Odds)
      extends Event
  final case class Updated(
              marketId: String,
              odds: Option[Odds],
              result: Option[Int]) extends Event
  final case class Closed(
              marketId: String,
              result: Int,
              at: OffsetDateTime) extends Event
  final case class Cancelled(
              marketId: String,
              reason: String) extends Event
  ...
}
```